Level 2 Assignment: Student Organizer App

Jetpack Compose UI Screens, Form Inputs, Custom Composables & Navigation (5 Screens)

Course: Modern Android App Development with Kotlin **Focus Area:** Jetpack Compose UI & Navigation

1. Assignment Overview & Objectives

In this assignment, you will design and build a multi-screen **Student Organizer Mobile Application** using Android Studio and Jetpack Compose. This project focuses strictly on declarative UI development, custom component encapsulation, form input state management, and multi-screen navigation using the Jetpack Compose Navigation component.

Note on Scope: This assignment strictly focuses on UI composables, layout arrangement, custom composables, state hoisting, form handling, and Jetpack Navigation. You are **NOT required** to implement Room Database persistence or network calls for this assignment. **All screen data should be represented using UI state variables or sample data models.**

Core Learning Outcomes

- **Declarative UI Layouts:** Master layout composables (Column, Row, Box, Scaffold, Card) to structure clean, responsive mobile interfaces.
- **Custom Composables & Reusability:** Design modular, reusable components with parameter contracts and Slot APIs (CourseInfo, AssignmentCard, StudentProfile, AppBar, BottomBar).
- **Form State Management & Validation:** Utilize remember and mutableStateOf to manage form inputs (OutlinedTextField, Checkbox, RadioButton, Dropdowns) and enforce input validation rules.
- **Multi-Screen Navigation:** Implement a centralized NavHostController, NavHost, and sealed class routes for seamless transition across 5 distinct screens.
- **State Hoisting & Event Callbacks:** Separate UI presentation from state logic by passing values and event callbacks down the composable tree.

2. Required Custom Composables & Parameter Contracts

To ensure clean architectural separation, students must implement the following reusable custom composables with strict parameter contracts:

Composable Name	Key Parameters	Responsibility / UI Purpose
AppBar	title: String, canNavigateBack: Boolean, onBackClick: () -> Unit	Top app bar with dynamic title and back navigation arrow when applicable.

BottomBar	navController: NavHostController, currentRoute: String?	Bottom navigation bar hosting tabs for Dashboard, Assignments, Profile, and Settings.
CourseInfo	courseName: String, courseCode: String, instructor: String, progress: Float	Card displaying active course details, instructor, and completion progress bar.
AssignmentCard	title: String, course: String, dueDate: String, priority: String, isCompleted: Boolean, onToggleComplete: () -> Unit	Interactive card for listing assignments with priority badges and completion checkbox.
StudentProfile	studentName: String, studentId: String, major: String, avatarRes: Int	Header card displaying student profile information and avatar graphic.
AssignmentForm	title: String, course: String, dueDate: String, priority: String, description: String, onTitleChange: (String) -> Unit, ...	State-hoisted reusable form component holding input fields for creating new assignments.

3. Screen Specifications & Structural Designs (5 Screens)

The Student Organizer App consists of 5 required UI screens. Each screen must strictly follow the structural layouts detailed below:

Screen 1: Dashboard Screen (`dashboard`)

Purpose: Serves as the main hub showing student overview, course list, and quick assignment highlights.
Layout Structure:
• Top Bar: Greeting header ('Welcome back, [Student Name]!').
• Quick Stats Row: Two elevated cards showing 'Active Courses (4)' and 'Pending Tasks (7)'.
• Active Courses Section: Horizontal scrollable row (`LazyRow`) displaying `CourseInfo` cards.
• Urgent Deadlines Section: Vertical list (`Column`) showing top 2 upcoming `AssignmentCard` items.
• Floating Action Button (FAB): A '+' FloatingActionButton that navigates directly to the Add Assignment Screen (`add_assignment`).

Screen 2: Assignments List Screen (`assignments`)

Purpose: Lists all student assignments with status filtering and quick completion toggles.
Layout Structure:
• Top Bar: 'My Assignments' AppBar.
• Filter Row: Horizontal row of filter RadioButtons ('All', 'Pending', 'Completed').
• Main Content: `LazyColumn` containing interactive `AssignmentCard` components.
• Floating Action Button (FAB): Prominent '+' button anchored at bottom-right that navigates to the Add Assignment Screen (`add_assignment`).

Screen 3: Profile Screen (`profile`)

Purpose: Displays student information and academic statistics.

Layout Structure:

• Top Bar: 'Student Profile' AppBar.

• Profile Header: Uses `StudentProfile` composable (Avatar, Name, Student ID, Major).

• Academic Stats Card: Displays GPA (3.82), Completed Credits (84), Current Semester (Fall 2026).

• Quick Actions: Buttons to 'Edit Profile' and 'Export Academic Summary'.

Screen 4: Settings Screen (`settings`)

Purpose: Allows users to configure application preferences.

Layout Structure:

• Top Bar: 'Settings' AppBar.

• Preferences List: `Column` containing toggleable preference items:

- Dark Mode Preference: `Row` with text label and `Switch` component.
- Push Notifications: `Row` with text label and `Checkbox` component.
- Default Start Screen: Radio button group (`Dashboard` vs `Assignments`).

Screen 5: Add Assignment Screen (`add_assignment`)

Purpose: A dedicated form input screen allowing students to enter and save a new assignment.

Layout Structure & Requirements:

• Top Bar: 'Add New Assignment' TopAppBar with Back Navigation arrow (`IconButton`) returning to previous screen.

• Form Container: LazyColumn

• Form Fields:

1. **Assignment Title:** `OutlinedTextField` (singleLine = true, required field).
2. **Course Name/Code:** `OutlinedTextField` or Dropdown selector (e.g. CS101, MATH201, ENG102).
3. **Due Date:** `OutlinedTextField` with trailing calendar icon.
4. **Priority Level:** Label + `Row` of `RadioButton` options ('Low', 'Medium', 'High').
5. **Description / Notes:** Multi-line `OutlinedTextField` (minLines = 3, maxLines = 5).
6. **Reminder Setting:** `Row` with `Checkbox` ('Enable Reminder Notification').

• **Action Buttons:** Row containing 'Cancel' (`OutlinedButton`) and 'Save Assignment' (`Button`). The Save button must be disabled until required fields (Title, Course, Due Date) are populated.

4. Navigation Architecture & Routing Table

Students must implement a central Navigation Graph utilizing `NavHostController` and `NavHost`. The 5 routes must be defined using a sealed class structure:

Route String	Target Composable Screen	Access Path / Trigger
dashboard	DashboardScreen	BottomBar tab click / App Launch initial destination

assignments	AssignmentsScreen	BottomBar tab click
profile	ProfileScreen	BottomBar tab click
settings	SettingsScreen	BottomBar tab click
add_assignment	AddAssignmentScreen	FAB '+' click on Dashboard or Assignments Screen / Back arrow returns to caller

5. Step-by-Step Implementation Tasks

- **Task 1: Navigation Setup & Routes:** Add `androidx.navigation:navigation-compose` dependency. Create a sealed class `Screen(val route: String)` defining all 5 routes (`dashboard`, `assignments`, `profile`, `settings`, `add_assignment`).
- **Task 2: Build Custom Composables:** Implement reusable components: `CourseInfo`, `AssignmentCard`, `StudentProfile`, `AppBar`, and `BottomBar` using appropriate parameters and Material Design styling.
- **Task 3: Build Core Screens (1-4):** Create `DashboardScreen`, `AssignmentsScreen`, `ProfileScreen`, and `SettingsScreen` layouts using Column, Row, Box, and LazyColumn.
- **Task 4: Build Add Assignment Form Screen (Screen 5):** Implement `AddAssignmentScreen` with text fields (`OutlinedTextField`), priority radio buttons (`RadioButton`), reminder checkbox (`Checkbox`), and action buttons.
- **Task 5: Form State Hoisting & Validation:** Manage form state using `remember { mutableStateOf() }`. Enable the 'Save Assignment' button only when required fields are non-empty. Handle back navigation on save/cancel.
- **Task 6: Wire Central Navigation & Scaffold:** Set up `rememberNavController()`, `NavHost`, and wrap main screens inside a `Scaffold` with `BottomBar` integration. Conditionally hide the BottomBar when on the `add_assignment` screen.